centralindia

Standard_D2as_v5

az vm list-skus --size Standard_B2ms --output table

Student Lab Manual

Azure VM, Kafka & .NET Microservices

Real-Time Payment Feedback with Bicep, Kafka KRaft and .NET 8

Estimated Duration: 2.5 hours

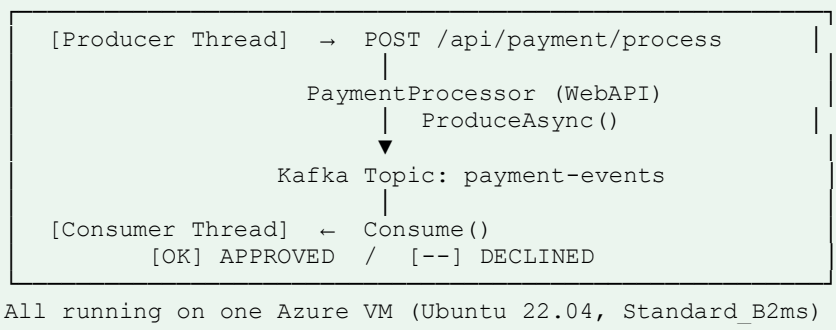
Lab Overview

In this lab you will provision a Linux VM on Azure using a Bicep template — an Infrastructure as Code approach that eliminates manual installation steps. Once the VM is ready, you will build two .NET 8 applications directly inside the VM:

- PaymentProcessor — an ASP.NET Core WebAPI that receives a payment request, randomly approves or declines it (like a mock payment gateway), and publishes the result to a Kafka topic.
- PaymentSimulator — a console app that continuously sends random payment requests to the WebAPI, and simultaneously reads results from Kafka, displaying them in real time with colour-coded output.

Architecture

Architecture



What You Will Learn

- How to write and deploy a Bicep template with a Custom Script Extension
- How Kafka works in KRaft mode (no ZooKeeper)
- How to produce and consume messages using Confluent.Kafka for .NET
- How the producer / consumer pattern enables real-time event streaming

Prerequisites Checklist

Confirm each item before starting:

- ☐ Azure subscription with Contributor role assigned
- ☐ Azure CLI installed and signed in — run az account show to confirm
- ☐ Windows Command Prompt (cmd.exe) or Windows Terminal
- ☐ SSH client available — test by typing ssh in cmd.exe (should show usage, not an error)
- ☐ A text editor — Notepad, VS Code, or any editor that can save plain text files

Part 1: Provision Infrastructure with Bicep

INFO

Bicep is Azure's Infrastructure as Code language. Instead of clicking through the portal, you describe your resources in a .bicep file and Azure creates them.

In this lab the Bicep template also attaches a Custom Script Extension — a mechanism that runs a bash script on the VM automatically after it is created. The script installs Java 17, .NET 8 SDK, and Apache Kafka 3.7. By the time deployment finishes, everything is installed and Kafka is already running as a background service.

Step 1.1 — Set Variables

Open a Command Prompt window. Set these variables — they are used in all subsequent commands.

CMD (Windows Command Prompt)

```
set RG=kafka-lab-rg
set LOCATION=eastus
set VM_NAME=kafka-lab-vm
set ADMIN_USER=azureuser
set ADMIN_PASS=KafkaLab@2024!
```

```
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export RG="kafka-lab-rg"
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export LOCATION="eastus"
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export VM_NAME="kafka-lab-vm"
 vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export ADMIN_USER="azureuser"

 vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export ADMIN_PASS="KafkaLab@2024!"

 dquote> export ADMIN_USER="azureuser"

[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % export ADMIN_PASS="KafkaLab@2024"

 vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % echo $RG
 echo $LOCATION
 echo $VM_NAME
 echo $ADMIN_USER
[echo $ADMIN_PASS
 kafka-lab-rg
 eastus
 kafka-lab-vm
 azureuser
 KafkaLab@2024
 vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC %
```

RECORD YOUR VALUES

RG =	kafka-lab-rg
LOCATION =	centralindia
ADMIN_PASS =	KafkaLab@2024

Step 1.2 — Create Resource Group

CMD (Windows Command Prompt)

```
az group create --name %RG% --location %LOCATION%
```

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab % az group create --name $RG --location $LOCATION
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/kafka-lab-rg",
  "location": "centralindia",
  "managedBy": null,
  "name": "kafka-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab %
```

Step 1.3 — Create Project Folder

CMD (Windows Command Prompt)

```
mkdir %USERPROFILE%\kafka-lab
cd %USERPROFILE%\kafka-lab
```

```
}
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 50 July 3 Azuer IaC % mkdir -p ~/kafka-lab
cd ~/kafka-lab
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab %
```

Step 1.4 — Create main.bicep

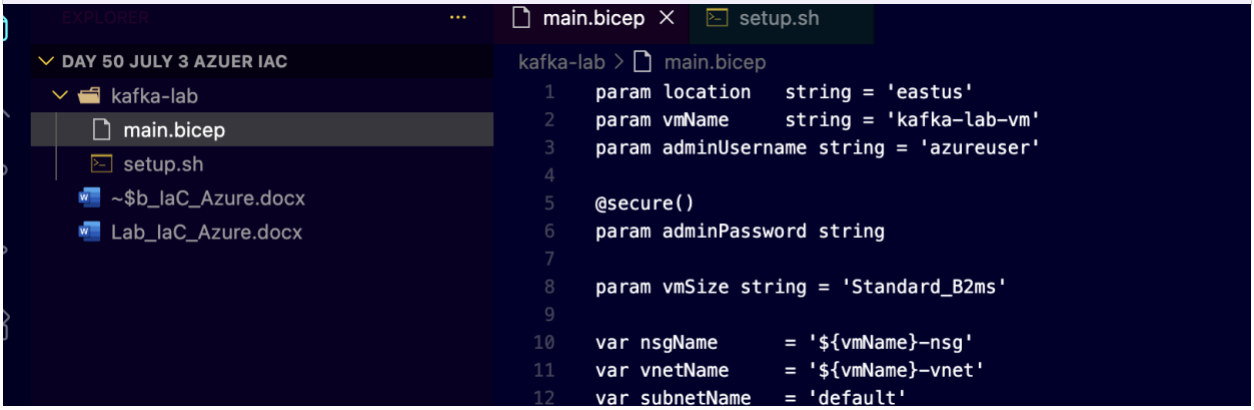
Open a text editor and save the following as main.bicep inside the kafka-lab folder:

BICEP (main.bicep)

```
param location    string = 'eastus'
param vmName      string = 'kafka-lab-vm'
param adminUsername string = 'azureuser'

@secure()
param adminPassword string
```

```
param vmSize string = 'Standard_B2ms'

var nsgName      = '${vmName}-nsg'
var vnetName     = '${vmName}-vnet'
var subnetName   = 'default'
var publicIpName = '${vmName}-pip'
var nicName      = '${vmName}-nic'

resource nsg 'Microsoft.Network/networkSecurityGroups@2023-04-01' = {
  name: nsgName
  location: location
  properties: {
    securityRules: [{
      name: 'AllowSSH'
      properties: {
        priority: 1000
        protocol: 'Tcp'
        access: 'Allow'
        direction: 'Inbound'
        sourceAddressPrefix: '*'
        sourcePortRange: '*'
        destinationAddressPrefix: '*'
        destinationPortRange: '22'
      }
    }]
  }
}

resource vnet 'Microsoft.Network/virtualNetworks@2023-04-01' = {
  name: vnetName
  location: location
  properties: {
    addressSpace: { addressPrefixes: ['10.0.0.0/16'] }
    subnets: [{
      name: subnetName
      properties: { addressPrefix: '10.0.0.0/24', networkSecurityGroup: { id:
nsg.id } }
    }]
  }
}

resource publicIp 'Microsoft.Network/publicIPAddresses@2023-04-01' = {
  name: publicIpName
  location: location
  sku: { name: 'Standard' }
  properties: { publicIPAllocationMethod: 'Static' }
}

resource nic 'Microsoft.Network/networkInterfaces@2023-04-01' = {
  name: nicName
  location: location
  properties: {
    ipConfigurations: [{
      name: 'ipconfig1'
      properties: {
        subnet:          { id: '${vnet.id}/subnets/${subnetName}' }
        publicIPAddress: { id: publicIp.id }
      }
    }]
  }
```

```
}

resource vm 'Microsoft.Compute/virtualMachines@2023-07-01' = {
  name: vmName
  location: location
  properties: {
    hardwareProfile: { vmSize: vmSize }
    osProfile: {
      computerName:  vmName
      adminUsername: adminUsername
      adminPassword: adminPassword
    }
    storageProfile: {
      imageReference: {
        publisher: 'Canonical'
        offer:     '0001-com-ubuntu-server-jammy'
        sku:       '22_04-lts-gen2'
        version:   'latest'
      }
      osDisk: {
        createOption: 'FromImage'
        managedDisk:  { storageAccountType: 'StandardSSD_LRS' }
      }
    }
    networkProfile: { networkInterfaces: [{ id: nic.id }] }
  }
}

resource setup 'Microsoft.Compute/virtualMachines/extensions@2023-07-01' = {
  parent: vm
  name: 'setup-kafka'
  location: location
  properties: {
    publisher:            'Microsoft.Azure.Extensions'
    type:                 'CustomScript'
    typeHandlerVersion:   '2.1'
    autoUpgradeMinorVersion: true
    settings: {}
    protectedSettings: {
      script: base64(loadTextContent('setup.sh'))
    }
  }
}

output publicIpAddress string = publicIp.properties.ipAddress
output sshCommand      string = 'ssh
${adminUsername}@${publicIp.properties.ipAddress}'
```



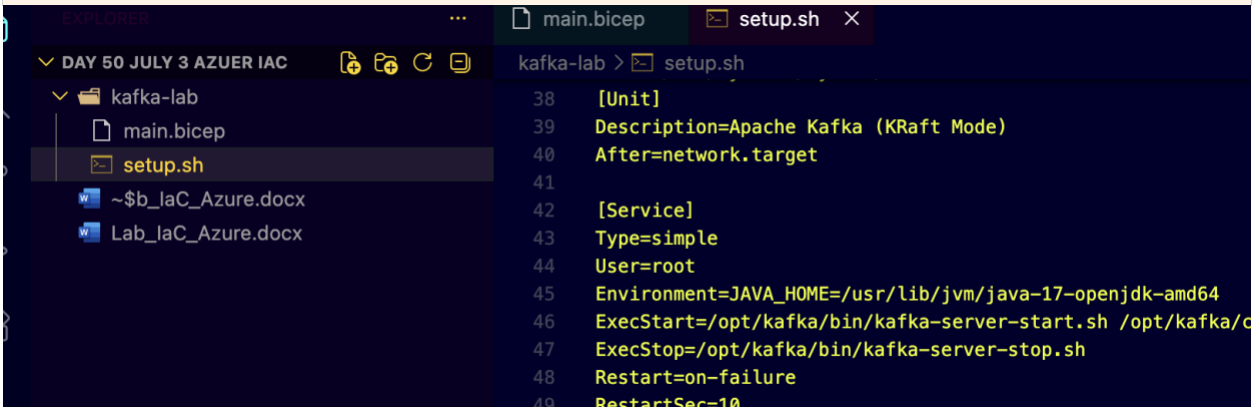
Step 1.5 — Create setup.sh

In the same kafka-lab folder, create a file called setup.sh with the following content. This is the script that installs everything on the VM automatically.

BASH SCRIPT (setup.sh)

```
#!/bin/bash
set -e
exec > /var/log/kafka-setup.log 2>&1

echo "=== Kafka Lab Setup Started ==="
date

# 1. System packages
apt-get update -y
apt-get install -y openjdk-17-jdk wget curl

# 2. Set JAVA_HOME system-wide
echo "JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64" >> /etc/environment
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64

# 3. Install .NET 8 SDK
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-
prod.deb \
     -O /tmp/ms-prod.deb
dpkg -i /tmp/ms-prod.deb
apt-get update -y
apt-get install -y dotnet-sdk-8.0

# 4. Download and extract Kafka 3.7.0
cd /opt
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz -O kafka.tgz
tar -xzf kafka.tgz
mv kafka_2.13-3.7.0 kafka
rm kafka.tgz

# 5. Format KRaft storage (single-node, no ZooKeeper)
KAFKA_UUID=$(JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-storage.sh random-uuid)
JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-storage.sh format \
  -t "$KAFKA_UUID" \
  -c /opt/kafka/config/kraft/server.properties

# 6. Create systemd service
cat > /etc/systemd/system/kafka.service << EOF
[Unit]
Description=Apache Kafka (KRaft Mode)
After=network.target

[Service]
Type=simple
User=root
Environment=JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
ExecStart=/opt/kafka/bin/kafka-server-start.sh
/opt/kafka/config/kraft/server.properties
ExecStop=/opt/kafka/bin/kafka-server-stop.sh
Restart=on-failure
RestartSec=10
```

```
[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable kafka
systemctl start kafka

# 7. Wait for Kafka then create the topic
MAX_WAIT=90; WAITED=0
until JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-topics.sh \
    --list --bootstrap-server localhost:9092 > /dev/null 2>&1; do
  [ $WAITED -ge $MAX_WAIT ] && echo "ERROR: Kafka timeout" && exit 1
  echo "Waiting for Kafka... (${WAITED}s elapsed)"
  sleep 5; WAITED=$((WAITED+5))
done

JAVA_HOME=$JAVA_HOME /opt/kafka/bin/kafka-topics.sh \
  --create --topic payment-events \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1

echo "=== Setup Complete ===" && date
```



Step 1.6 — Deploy the Bicep Template

Make sure you are in the kafka-lab folder, then run:

CMD (Windows Command Prompt)

```
cd %USERPROFILE%\kafka-lab

az deployment group create ^
  --resource-group %RG% ^
  --name kafka-deploy ^
  --template-file main.bicep ^
  --parameters adminPassword="%ADMIN_PASS%"
```

INFO

This deployment takes approximately 10-14 minutes. Azure is:

1. Creating the VM, VNet, NSG, NIC and public IP (~2-3 min)
2. Running setup.sh via the Custom Script Extension (~8-10 min)
   setup.sh installs Java, .NET SDK, and Kafka — then starts the Kafka service.

You will see "Running..." in the terminal while it works. When it finishes, the output section shows publicIpAddress and sshCommand.

Step 1.7 — Capture the VM IP Address

When deployment finishes, capture the public IP from the outputs:

CMD (Windows Command Prompt)

```
for /f "delims=" %a in ('az deployment group show ^
  --resource-group %RG% --name kafka-deploy ^
  --query "properties.outputs.publicIpAddress.value" -o tsv') do set VM_IP=%a

echo VM IP: %VM_IP%
```

RECORD YOUR VALUES

VM_IP =	74.225.170.255

```
}
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab % export VM_IP=$(az deployment group show \
  --resource-group $RG --name kafka-deploy \
  --query "properties.outputs.publicIpAddress.value" -o tsv)
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab % echo "VM IP: $VM_IP"
VM IP: 74.225.170.255
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab %
```

Step 1.8 — Connect to the VM via SSH

CMD (Windows Command Prompt)

```
ssh %ADMIN_USER%@%VM_IP%

(Type yes to accept the host key fingerprint)
(Enter password: KafkaLab@2024)
```

INFO

You are now INSIDE the Linux VM. Your prompt changes to:

azureuser@kafka-lab-vm:~$

Everything from here until Part 4 runs in this SSH session.

Linux commands are case-sensitive — type them exactly as shown.

```
VM IP: 74.225.170.255
vaibhavgupta@FVFG905WQ05F-vaibhavgupta kafka-lab % ssh $ADMIN_USER@$VM_IP
The authenticity of host '74.225.170.255 (74.225.170.255)' can't be established.
ED25519 key fingerprint is: SHA256:xG3SIvYNqxXFfdLV8yotvgZ041WP0z5yVObPoPfS2U4
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '74.225.170.255' (ED25519) to the list of known hosts.
azureuser@74.225.170.255's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1059-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Jul  3 10:51:09 UTC 2026

  System load:  0.08              Processes:             114
  Usage of /:   11.7% of 28.89GB  Users logged in:       0
  Memory usage: 10%               IPv4 address for eth0: 10.0.0.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

26 updates can be applied immediately.
25 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@kafka-lab-vm:~$
```

Step 1.9 — Verify the Stack

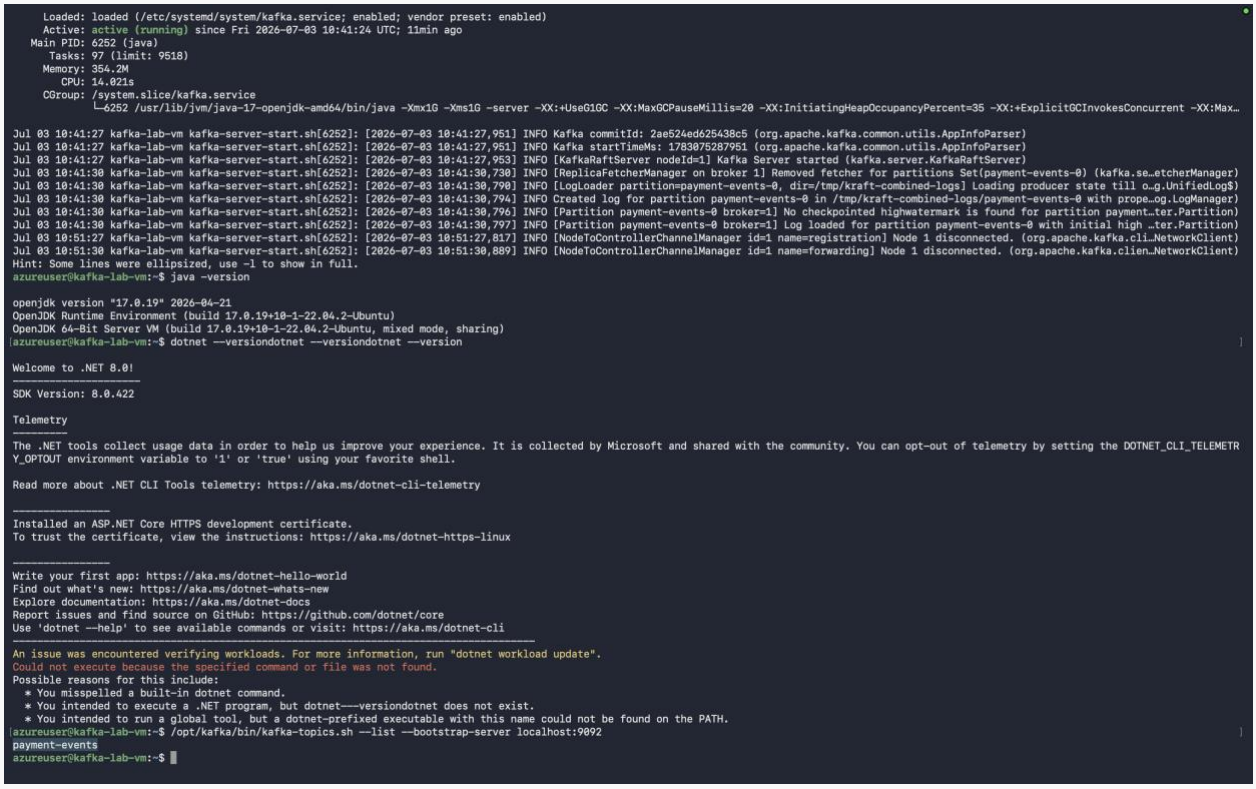
Confirm Java, .NET, and Kafka are all ready:

BASH (SSH — Linux VM)

```
# Check Kafka service status
systemctl status kafka --no-pager

# Check Java version
java -version

# Check .NET version
dotnet --version

# List Kafka topics — should show: payment-events
/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

RECORD YOUR VALUES

Java version:	openjdk version "17.0.19" 2026-04-21
.NET version:	SDK Version: 8.0.422
Kafka topics listed:	payment-events



REFLECTION QUESTIONS

1. What does loadTextContent('setup.sh') do in the Bicep template? When does it run?

loadTextContent('setup.sh') reads the contents of the setup.sh file and includes it in the Bicep template during deployment

2. What is the role of the Custom Script Extension? How does Azure know what to run?

The Custom Script Extension runs a script on the Azure VM, and Azure knows what to run from the command specified in the extension settings.

3. Why does KRaft mode need kafka-storage.sh format to run before Kafka can start?

KRaft mode needs **kafka-storage.sh format** to initialize the Kafka storage before the broker can start.

Part 2: Build the PaymentProcessor WebAPI

This WebAPI acts as a mock payment gateway. It receives a payment request, randomly approves (~70%) or declines (~30%) it, publishes the result to Kafka, and returns a JSON response.

Step 2.1 — Create the WebAPI Project

BASH (SSH — Linux VM)

```
cd ~
dotnet new webapi -n PaymentProcessor --no-https
cd PaymentProcessor
```

```
azureuser@kafka-lab-vm:~$ cd ~
dotnet new webapi -n PaymentProcessor --no-https
cd PaymentProcessor
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring /home/azureuser/PaymentProcessor/PaymentProcessor.csproj:
  Determining projects to restore...
  Restored /home/azureuser/PaymentProcessor/PaymentProcessor.csproj (in 2.24 sec).
Restore succeeded.


azureuser@kafka-lab-vm:~/PaymentProcessor$
```

Step 2.2 — Add Confluent.Kafka Package

BASH (SSH — Linux VM)

```
dotnet add package Confluent.Kafka
```

```
azureuser@kafka-lab-vm:~/PaymentProcessor$ dotnet add package Confluent.Kafka
  Determining projects to restore...
  Writing /tmp/tmpIarFxH.tmp
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/share/dotnet/sdk/8.0.422/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/share/dotnet/sdk/8.0.422/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Confluent.Kafka' into project '/home/azureuser/PaymentProcessor/PaymentProcessor.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/index.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/index.json 457ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/0.0.1/1.0.0-experimental-10.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/0.0.1/1.0.0-experimental-10.json 217ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.0.0-experimental-11/1.7.0.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.0.0-experimental-11/1.7.0.json 211ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.8.0/2.14.2-rc1.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.8.0/2.14.2-rc1.json 208ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/2.14.2/2.15.0.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/2.14.2/2.15.0.json 213ms
info : Restoring packages for /home/azureuser/PaymentProcessor/PaymentProcessor.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/confluent.kafka/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/confluent.kafka/index.json 20ms
info :   GET https://api.nuget.org/v3-flatcontainer/confluent.kafka/2.15.0/confluent.kafka.2.15.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/confluent.kafka/2.15.0/confluent.kafka.2.15.0.nupkg 5ms
info :   GET https://api.nuget.org/v3-flatcontainer/librdkafka.redist/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/librdkafka.redist/index.json 5ms
info :   GET https://api.nuget.org/v3-flatcontainer/librdkafka.redist/2.15.0/librdkafka.redist.2.15.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/librdkafka.redist/2.15.0/librdkafka.redist.2.15.0.nupkg 6ms
info : Installed Confluent.Kafka 2.15.0 from https://api.nuget.org/v3/index.json to /home/azureuser/.nuget/packages/confluent.kafka/2.15.0 with content hash /BOEnIKAq1AHq6vPl+0RBDiKwI6L6tJaYBo1qH+OAUP5MFy0i0MhSfgtqKW9/M8J9dQPfb/izWJ+R0Uc6DyW2w==.
info : Installed librdkafka.redist 2.15.0 from https://api.nuget.org/v3/index.json to /home/azureuser/.nuget/packages/librdkafka.redist/2.15.0 with content hash DzG2YcqWPT3Ly91aeDYsgsVcJXlOMiK9oqyRdPvQz9ZMbcV8Ob0rQA6fNUQ4sarmzFk4phqwPlVQxtZDGLP69w==.
info :   CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/vulnerability.base.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/2026.07.03.06.08.37/vulnerability.update.json
info : Package 'Confluent.Kafka' is compatible with all the specified frameworks in project '/home/azureuser/PaymentProcessor/PaymentProcessor.csproj'.
info : PackageReference for package 'Confluent.Kafka' version '2.15.0' added to file '/home/azureuser/PaymentProcessor/PaymentProcessor.csproj'.
info : Writing assets file to disk. Path: /home/azureuser/PaymentProcessor/obj/project.assets.json
log  : Restored /home/azureuser/PaymentProcessor/PaymentProcessor.csproj (in 1.5 sec).
azureuser@kafka-lab-vm:~/PaymentProcessor$
```

Step 2.3 — Create the Models

BASH (SSH — Linux VM)

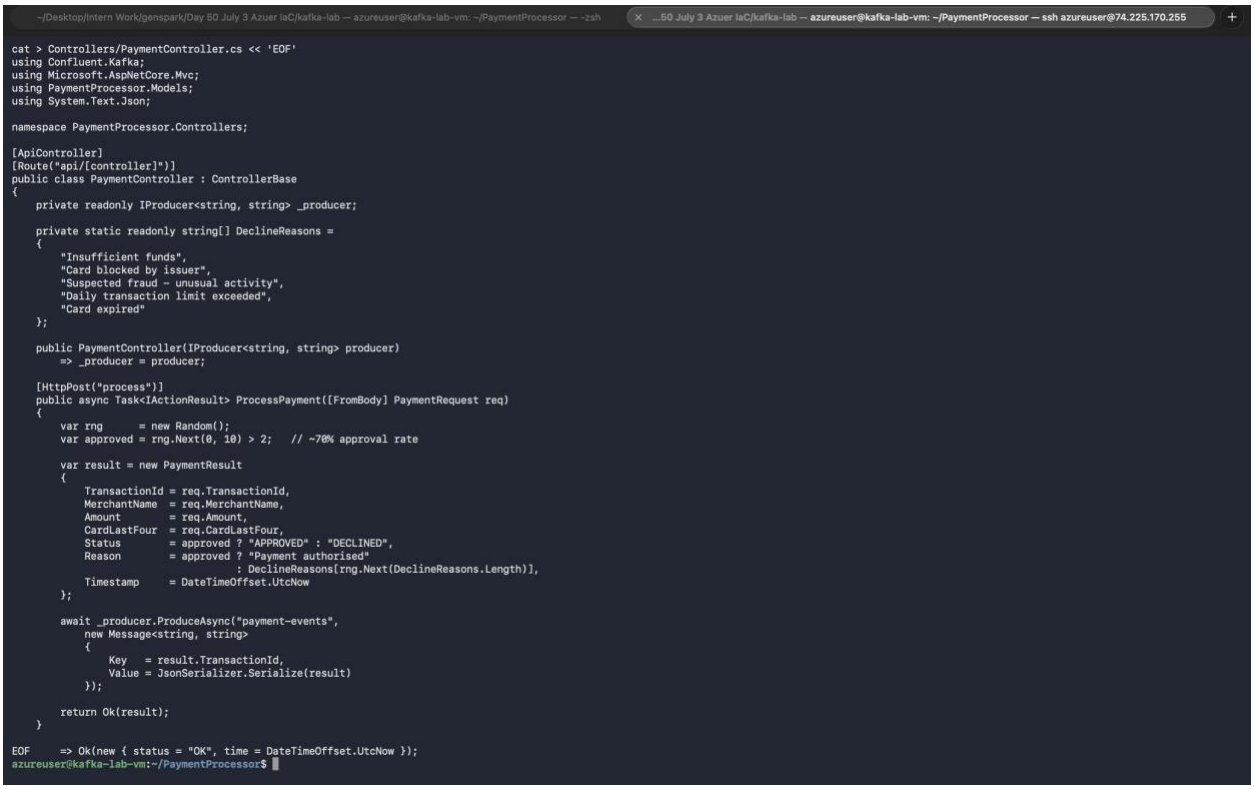
```
mkdir -p Models

cat > Models/PaymentRequest.cs << 'EOF'
namespace PaymentProcessor.Models;

public class PaymentRequest
{
    public string TransactionId  { get; set; } = string.Empty;
    public string MerchantName   { get; set; } = string.Empty;
    public decimal Amount        { get; set; }
    public string CardLastFour   { get; set; } = string.Empty;
}
EOF

cat > Models/PaymentResult.cs << 'EOF'
namespace PaymentProcessor.Models;

public class PaymentResult
{
    public string TransactionId  { get; set; } = string.Empty;
    public string MerchantName   { get; set; } = string.Empty;
    public decimal Amount        { get; set; }
    public string CardLastFour   { get; set; } = string.Empty;
    public string Status         { get; set; } = string.Empty;
    public string Reason         { get; set; } = string.Empty;
    public DateTimeOffset Timestamp { get; set; }
}
EOF
```

Step 2.4 — Create the Payment Controller

BASH (SSH — Linux VM)

```
mkdir -p Controllers

cat > Controllers/PaymentController.cs << 'EOF'
using Confluent.Kafka;
using Microsoft.AspNetCore.Mvc;
using PaymentProcessor.Models;
using System.Text.Json;

namespace PaymentProcessor.Controllers;

[ApiController]
```

```
[Route("api/[controller]")]
public class PaymentController : ControllerBase
{
    private readonly IProducer<string, string> _producer;

    private static readonly string[] DeclineReasons =
    {
        "Insufficient funds",
        "Card blocked by issuer",
        "Suspected fraud - unusual activity",
        "Daily transaction limit exceeded",
        "Card expired"
    };

    public PaymentController(IProducer<string, string> producer)
        => _producer = producer;

    [HttpPost("process")]
    public async Task<IActionResult> ProcessPayment([FromBody] PaymentRequest req)
    {
        var rng      = new Random();
        var approved = rng.Next(0, 10) > 2;   // ~70% approval rate

        var result = new PaymentResult
        {
            TransactionId = req.TransactionId,
            MerchantName  = req.MerchantName,
            Amount        = req.Amount,
            CardLastFour  = req.CardLastFour,
            Status        = approved ? "APPROVED" : "DECLINED",
            Reason        = approved ? "Payment authorised"
                                     :
DeclineReasons[rng.Next(DeclineReasons.Length)],
            Timestamp     = DateTimeOffset.UtcNow
        };

        await _producer.ProduceAsync("payment-events",
            new Message<string, string>
            {
                Key   = result.TransactionId,
                Value = JsonSerializer.Serialize(result)
            });

        return Ok(result);
    }

    [HttpGet("health")]
    public IActionResult Health()
        => Ok(new { status = "OK", time = DateTimeOffset.UtcNow });
}
EOF
```



Step 2.5 — Replace Program.cs

BASH (SSH — Linux VM)

```
cat > Program.cs << 'EOF'
using Confluent.Kafka;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

var producerConfig = new ProducerConfig { BootstrapServers = "localhost:9092" };
builder.Services.AddSingleton<IProducer<string, string>>(
    new ProducerBuilder<string, string>(producerConfig).Build());

var app = builder.Build();
app.MapControllers();
app.Run("http://localhost:5100");
EOF
```

Step 2.6 — Remove Default Template Files

BASH (SSH — Linux VM)

```
rm -f WeatherForecast.cs Controllers/WeatherForecastController.cs
```

Step 2.7 — Build and Run

BASH (SSH — Linux VM)

```
dotnet run
```

You should see:

Expected Output

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

INFO

Leave this terminal running. The WebAPI must stay active for Part 3.

You will open a NEW SSH session for the console app.

REFLECTION QUESTIONS

1. Why is IProducer<string, string> registered as a Singleton (not Scoped or Transient)?

IProducer<string, string> is registered as a Singleton because one producer instance can be shared by the whole application

2. What do the two type parameters of IProducer<string, string> represent?

The first string is the message key, and the second string is the message value.

3. The controller writes to Kafka after determining the result. Why might a real system

A real system may write to Kafka first to make sure the event is safely recorded before responding.

4. write to Kafka BEFORE sending the HTTP response?

Writing to Kafka before the HTTP response helps prevent data loss and ensures other services can process the event.

Part 3: Build the PaymentSimulator Console App

The simulator has two concurrent tasks running in parallel. One task sends a payment request to the WebAPI every 3 seconds. The other task listens on the Kafka topic and prints each payment result as it arrives — in real time.

Step 3.1 — Open a Second SSH Session

Leave the WebAPI terminal open. Open a new Command Prompt window on your machine and SSH in again:

CMD (Windows Command Prompt)

```
ssh %ADMIN_USER%@%VM_IP%

(If %VM_IP% is no longer set, re-run the capture command from Step 1.7)
```

Step 3.2 — Create the Console Project

BASH (SSH — Linux VM)

```
cd ~
dotnet new console -n PaymentSimulator
cd PaymentSimulator
```

Step 3.3 — Add Confluent.Kafka Package

BASH (SSH — Linux VM)

```
dotnet add package Confluent.Kafka
```

```
azureuser@kafka-lab-vm:~/PaymentSimulator$ dotnet add package Confluent.Kafka
  Determining projects to restore...
  Writing /tmp/tmp8dSX54.tmp
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/share/dotnet/sdk/8.0.422/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/share/dotnet/sdk/8.0.422/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Confluent.Kafka' into project '/home/azureuser/PaymentSimulator/PaymentSimulator.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/index.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/index.json 229ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/0.0.1/1.0.0-experimental-10.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/0.0.1/1.0.0-experimental-10.json 220ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.0.0-experimental-11/1.7.0.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.0.0-experimental-11/1.7.0.json 219ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.8.0/2.14.2-rc1.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/1.8.0/2.14.2-rc1.json 243ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/2.14.2/2.15.0.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/confluent.kafka/page/2.14.2/2.15.0.json 230ms
info : Restoring packages for /home/azureuser/PaymentSimulator/PaymentSimulator.csproj...
info :   GET https://api.nuget.org/v3/vulnerabilities/index.json
info :   OK https://api.nuget.org/v3/vulnerabilities/index.json 230ms
info :   GET https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/vulnerability.base.json
info :   GET https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/2026.07.03.06.08.37/vulnerability.update.json
info :   OK https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/vulnerability.base.json 208ms
info :   OK https://api.nuget.org/v3-vulnerabilities/2026.07.03.06.08.37/2026.07.03.06.08.37/vulnerability.update.json 236ms
info : Package 'Confluent.Kafka' is compatible with all the specified frameworks in project '/home/azureuser/PaymentSimulator/PaymentSimulator.csproj'.
info : PackageReference for package 'Confluent.Kafka' version '2.15.0' added to file '/home/azureuser/PaymentSimulator/PaymentSimulator.csproj'.
info : Writing assets file to disk. Path: /home/azureuser/PaymentSimulator/obj/project.assets.json
log  : Restored /home/azureuser/PaymentSimulator/PaymentSimulator.csproj (in 645 ms).
azureuser@kafka-lab-vm:~/PaymentSimulator$
```

Step 3.4 — Replace Program.cs

BASH (SSH — Linux VM)

```
cat > Program.cs << 'EOF'
using Confluent.Kafka;
using System.Net.Http.Json;
using System.Text.Json;

const string WebApiUrl = "http://localhost:5100";
const string KafkaBoot = "localhost:9092";
const string TopicName = "payment-events";

string[] Merchants = { "Amazon UK", "Tesco Online", "Netflix",
                       "Uber Eats", "Sainsburys", "ASOS", "Spotify" };
string[] Cards = { "4242", "1234", "5678", "9999", "0011" };

var cts        = new CancellationTokenSource();
var httpClient = new HttpClient { BaseAddress = new Uri(WebApiUrl) };
var rng        = new Random();
int counter    = 0;

Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

// ── Kafka Consumer (background) ──────────────────────────
var consumerTask = Task.Run(() =>
{
    var cfg = new ConsumerConfig
    {
        BootstrapServers = KafkaBoot,
        GroupId          = "payment-simulator-group",
        AutoOffsetReset  = AutoOffsetReset.Latest
    };
    using var consumer = new ConsumerBuilder<string, string>(cfg).Build();
    consumer.Subscribe(TopicName);

    Console.ForegroundColor = ConsoleColor.Cyan;
    Console.WriteLine($"[Consumer] Subscribed to '{TopicName}'. Waiting...\n");
    Console.ResetColor();

    while (!cts.Token.IsCancellationRequested)
    {
        try
        {
```

```
            var cr = consumer.Consume(TimeSpan.FromSeconds(1));
            if (cr is null) continue;

            var r = JsonSerializer.Deserialize<PaymentResult>(
                cr.Message.Value,
                new JsonSerializerOptions { PropertyNameCaseInsensitive = true
})!;

            Console.ForegroundColor = r.Status == "APPROVED"
                ? ConsoleColor.Green : ConsoleColor.Red;

            Console.WriteLine(
                $"  {(r.Status == "APPROVED" ? "[OK]" : "[--]")} " +
                $"[{r.Timestamp:HH:mm:ss}]  {r.TransactionId}  " +
                $"{r.MerchantName,-16} £{r.Amount,7:F2}  " +
                $"****{r.CardLastFour}  {r.Status,-8}  {r.Reason}");

            Console.ResetColor();
        }
        catch (ConsumeException ex)
            when (!cts.Token.IsCancellationRequested)
        {
            Console.Error.WriteLine($"[Consumer error] {ex.Error.Reason}");
        }
    }
    consumer.Close();
});

await Task.Delay(2000);

Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("================================================");
Console.WriteLine("  Payment Simulator  --  Real-Time Feed      ");
Console.WriteLine("================================================\n");
Console.ResetColor();
Console.WriteLine("Sending one payment every 3 seconds. Ctrl+C to stop.\n");

// ── HTTP Producer loop ──────────────────────────────────
while (!cts.Token.IsCancellationRequested)
{
    var req = new {
        transactionId = $"TXN{++counter:D6}",
        merchantName  = Merchants[rng.Next(Merchants.Length)],
        amount        = Math.Round((decimal)(rng.NextDouble() * 249 + 1), 2),
        cardLastFour  = Cards[rng.Next(Cards.Length)]
    };

    Console.ForegroundColor = ConsoleColor.DarkYellow;
    Console.WriteLine($"-> [{DateTime.UtcNow:HH:mm:ss}] {req.transactionId}  " +
        $"{req.merchantName,-16} £{req.amount:F2}  ****{req.cardLastFour}");
    Console.ResetColor();

    try { await httpClient.PostAsJsonAsync("/api/payment/process", req,
cts.Token); }
    catch (Exception ex) when (!cts.Token.IsCancellationRequested)
        { Console.Error.WriteLine($"[HTTP error] {ex.Message}"); }

    await Task.Delay(3000).ContinueWith(_ => { });
}
```

```
await consumerTask;
Console.WriteLine("\nSimulator stopped.");

record PaymentResult(string TransactionId, string MerchantName,
    decimal Amount, string CardLastFour, string Status,
    string Reason, DateTimeOffset Timestamp);
EOF
```

Step 3.5 — Run the Simulator

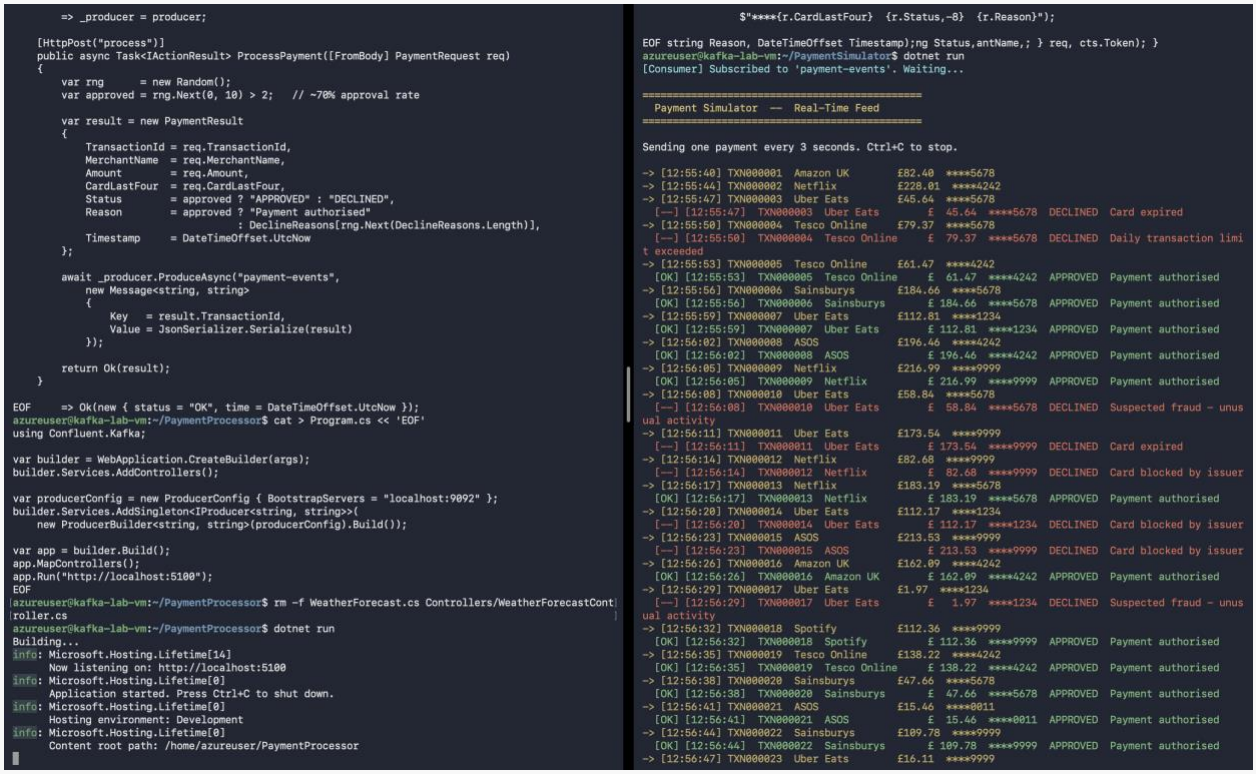
BASH (SSH — Linux VM)

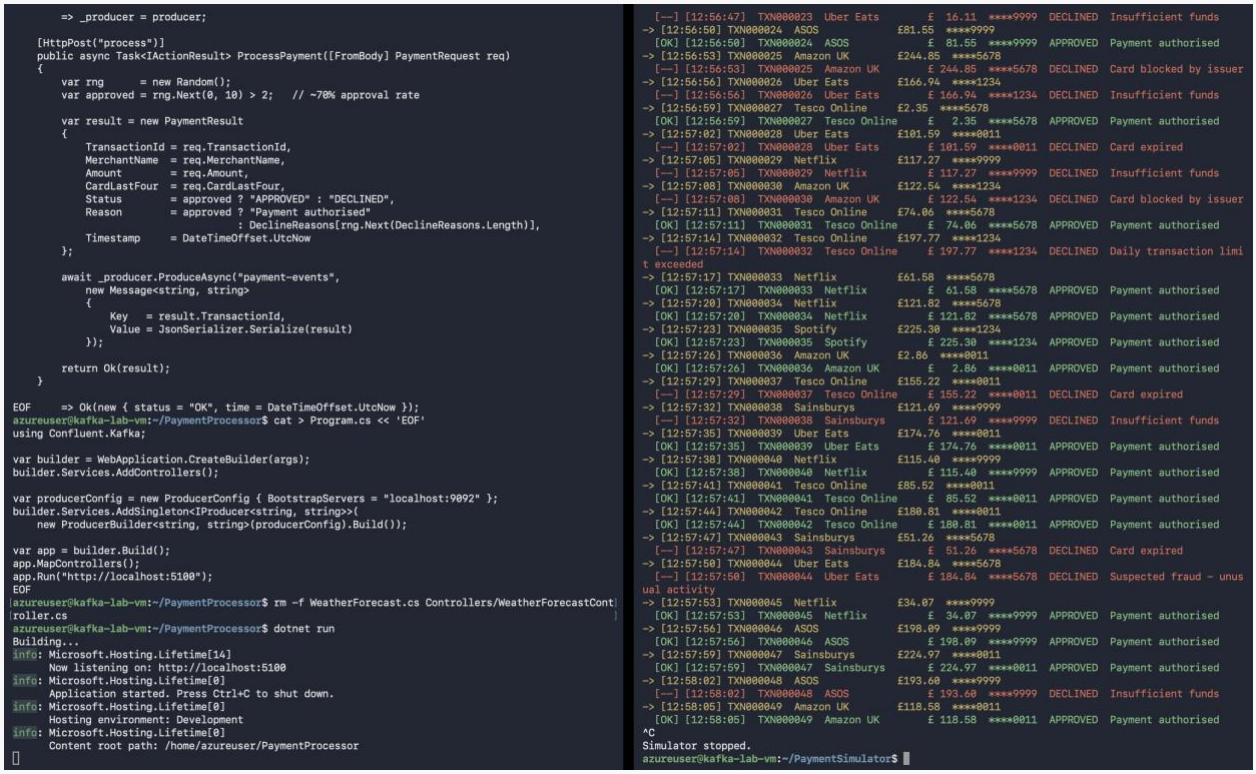
```
dotnet run
```

You will see outbound arrows (->) as payments are sent, and [OK]/[--] lines as results arrive from Kafka:

Expected Output (approximate)

```
================================================
  Payment Simulator  --  Real-Time Feed
================================================

[Consumer] Subscribed to payment-events. Waiting...

-> [14:23:01] TXN000001  Amazon UK          £127.45  ****4242
-> [14:23:04] TXN000002  Netflix             £89.99  ****1234

  [OK] [14:23:01]  TXN000001  Amazon UK          £127.45  ****4242  APPROVED
Payment authorised
  [--] [14:23:04]  TXN000002  Netflix             £89.99  ****1234  DECLINED
Insufficient funds
```


=> _producer = producer;
[HttpPost("process")]
public async Task<IActionResult> ProcessPayment([FromBody] PaymentRequest req)
{
var rng = new Random();
var approved = rng.Next(0, 10) > 2; // ~70% approval rate
var result = new PaymentResult
{
TransactionId = req.TransactionId,
MerchantName = req.MerchantName,
Amount = req.Amount,
CardLastFour = req.CardLastFour,
Status = approved ? "APPROVED" : "DECLINED",
Reason = approved ? "Payment authorised"
: DeclineReasons[rng.Next(DeclineReasons.Length)],
Timestamp = DateTimeOffset.UtcNow
};
await _producer.ProduceAsync("payment-events",
new Message<string, string>
{
Key = result.TransactionId,
Value = JsonSerializer.Serialize(result)
});
return Ok(result);
}
EOF => Ok(new { status = "OK", time = DateTimeOffset.UtcNow });
azureuser@kafka-lab-vm:~/PaymentProcessor$ cat > Program.cs << 'EOF'
using Confluent.Kafka;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
var producerConfig = new ProducerConfig { BootstrapServers = "localhost:9092" };
builder.Services.AddSingleton<IProducer<string, string>>(
new ProducerBuilder<string, string>(producerConfig).Build());
var app = builder.Build();
app.MapControllers();
app.Run("http://localhost:5100");
EOF
azureuser@kafka-lab-vm:~/PaymentProcessor$ rm -f WeatherForecast.cs Controllers/WeatherForecastController.cs
azureuser@kafka-lab-vm:~/PaymentProcessor$ dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: /home/azureuser/PaymentProcessor
$"****{r.CardLastFour} {r.Status,-8} {r.Reason}");
EOF string Reason, DateTimeOffset Timestamp);ng Status,antName,; } req, cts.Token); }
azureuser@kafka-lab-vm:~/PaymentSimulator$ dotnet run
[Consumer] Subscribed to 'payment-events'. Waiting...
Payment Simulator — Real-Time Feed
Sending one payment every 3 seconds. Ctrl+C to stop.
-> [12:55:40] TXN000001 Amazon UK £82.40 ****5678
-> [12:55:44] TXN000002 Netflix £228.01 ****4242
-> [12:55:47] TXN000003 Uber Eats £45.64 ****5678
[—] [12:55:47] TXN000003 Uber Eats £ 45.64 ****5678 DECLINED Card expired
-> [12:55:50] TXN000004 Tesco Online £79.37 ****5678
[—] [12:55:50] TXN000004 Tesco Online £ 79.37 ****5678 DECLINED Daily transaction limit exceeded
-> [12:55:53] TXN000005 Tesco Online £61.47 ****4242
[OK] [12:55:53] TXN000005 Tesco Online £ 61.47 ****4242 APPROVED Payment authorised
-> [12:55:56] TXN000006 Sainsburys £184.66 ****5678
[OK] [12:55:56] TXN000006 Sainsburys £ 184.66 ****5678 APPROVED Payment authorised
-> [12:55:59] TXN000007 Uber Eats £112.81 ****1234
[OK] [12:55:59] TXN000007 Uber Eats £ 112.81 ****1234 APPROVED Payment authorised
-> [12:56:02] TXN000008 ASOS £196.46 ****4242
[OK] [12:56:02] TXN000008 ASOS £ 196.46 ****4242 APPROVED Payment authorised
-> [12:56:05] TXN000009 Netflix £216.99 ****9999
[OK] [12:56:05] TXN000009 Netflix £ 216.99 ****9999 APPROVED Payment authorised
-> [12:56:08] TXN000010 Uber Eats £58.84 ****5678
[—] [12:56:08] TXN000010 Uber Eats £ 58.84 ****5678 DECLINED Suspected fraud — unusual activity
-> [12:56:11] TXN000011 Uber Eats £173.54 ****9999
[—] [12:56:11] TXN000011 Uber Eats £ 173.54 ****9999 DECLINED Card expired
-> [12:56:14] TXN000012 Netflix £82.68 ****9999
[—] [12:56:14] TXN000012 Netflix £ 82.68 ****9999 DECLINED Card blocked by issuer
-> [12:56:17] TXN000013 Netflix £183.19 ****5678
[OK] [12:56:17] TXN000013 Netflix £ 183.19 ****5678 APPROVED Payment authorised
-> [12:56:20] TXN000014 Uber Eats £112.17 ****1234
[—] [12:56:20] TXN000014 Uber Eats £ 112.17 ****1234 DECLINED Card blocked by issuer
-> [12:56:23] TXN000015 ASOS £213.53 ****9999
[—] [12:56:23] TXN000015 ASOS £ 213.53 ****9999 DECLINED Card blocked by issuer
-> [12:56:26] TXN000016 Amazon UK £162.09 ****4242
[OK] [12:56:26] TXN000016 Amazon UK £ 162.09 ****4242 APPROVED Payment authorised
-> [12:56:29] TXN000017 Uber Eats £1.97 ****1234
[—] [12:56:29] TXN000017 Uber Eats £ 1.97 ****1234 DECLINED Suspected fraud — unusual activity
-> [12:56:32] TXN000018 Spotify £112.36 ****9999
[OK] [12:56:32] TXN000018 Spotify £ 112.36 ****9999 APPROVED Payment authorised
-> [12:56:35] TXN000019 Tesco Online £138.22 ****4242
[OK] [12:56:35] TXN000019 Tesco Online £ 138.22 ****4242 APPROVED Payment authorised
-> [12:56:38] TXN000020 Sainsburys £47.66 ****5678
[OK] [12:56:38] TXN000020 Sainsburys £ 47.66 ****5678 APPROVED Payment authorised
-> [12:56:41] TXN000021 ASOS £15.46 ****0011
[OK] [12:56:41] TXN000021 ASOS £ 15.46 ****0011 APPROVED Payment authorised
-> [12:56:44] TXN000022 Sainsburys £109.78 ****9999
[OK] [12:56:44] TXN000022 Sainsburys £ 109.78 ****9999 APPROVED Payment authorised
-> [12:56:47] TXN000023 Uber Eats £16.11 ****9999



REFLECTION QUESTIONS

1. Why does the [OK] / [--] result appear slightly after the -> sending line?

__because Kafka processes the message in the background after it is sent.

2. What does AutoOffsetReset.Latest mean? What would happen if you used Earliest instead?

AutoOffsetReset.Latest reads only new messages, while Earliest reads all old messages from the beginning

3. If you stopped the WebAPI and restarted it while the simulator was running,

it reconnects to Kafka and continues consuming messages

4. what would you expect to see? What does this tell you about Kafka vs a direct API call?

Kafka stores messages, so they are not lost if the WebAPI is down, unlike a direct API call which would fail.

5. In a real payment system, what downstream services might consume from this same topic?

Services like billing, fraud detection, notifications, reporting, and analytics can consume the same Kafka topic.

Common Errors & Fixes

Error / Symptom	Cause	Fix
kafka-topics.sh: Connection refused	Kafka not started or JAVA_HOME missing	Run: export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 Then: systemctl restart kafka Wait 15s and retry
Deployment extension shows Failed in portal	setup.sh errored during deployment	SSH in and run: cat /var/log/kafka-setup.log Find the last error line and fix
dotnet: command not found	.NET SDK not yet installed (deployment still running)	Run: tail -f /var/log/kafka-setup.log Wait for "Setup Complete" then retry
HTTP error: Connection refused in simulator	WebAPI not running or wrong port	Confirm terminal 1 shows "Listening on http://localhost:5100"
InvalidTemplateDeployment on az deployment create	setup.sh not in same folder as main.bicep	Both files must be in %USERPROFILE%\kafka-lab. Run the deploy command from that folder
Kafka service: failed (Exit code 1)	Storage not formatted before first start	Run: journalctl -u kafka -n 20 --no-pager Re-run kafka-storage.sh format if needed

Cleanup

When you are finished, stop the applications and delete all Azure resources:

Stop the Applications

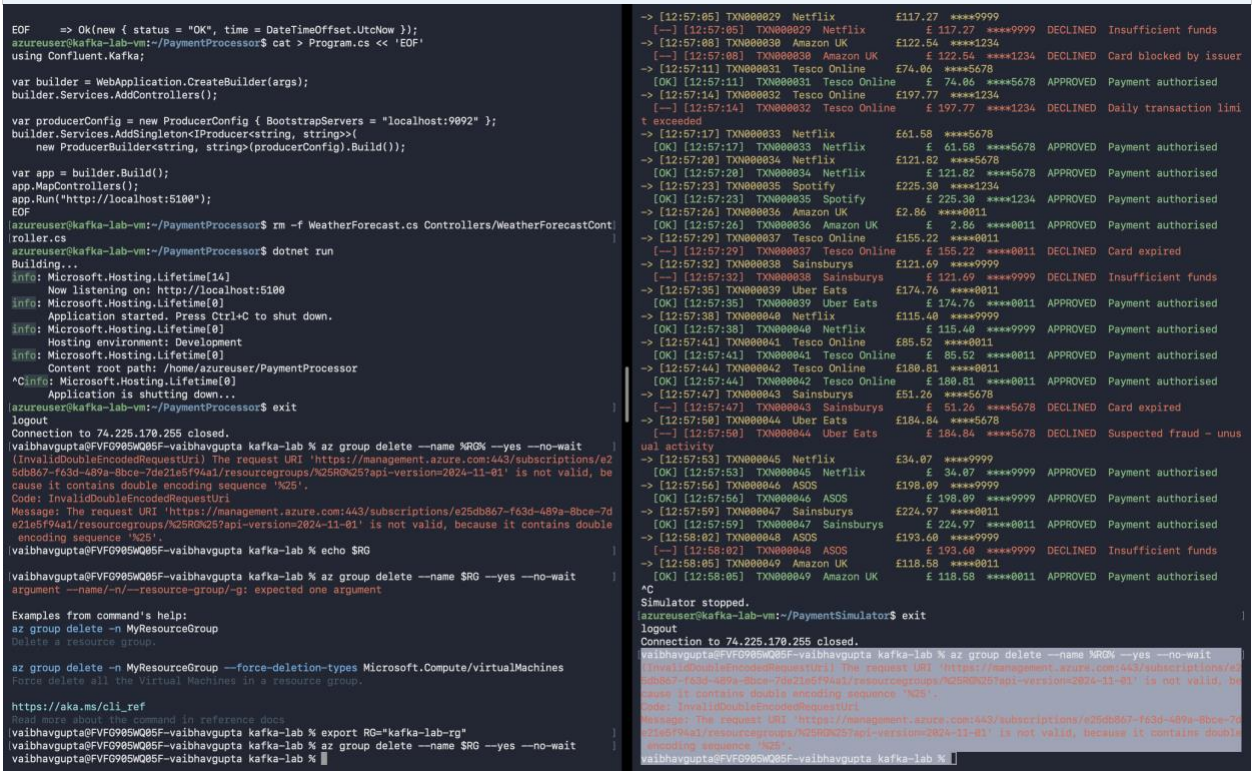
BASH (SSH — Linux VM)

```
# In each SSH terminal, press Ctrl+C to stop the running app, then:
exit
```

Delete All Azure Resources

CMD (Windows Command Prompt)

```
az group delete --name %RG% --yes --no-wait
```



INFO

This deletes the VM, disk, NIC, public IP, VNet, and NSG in one command.

The Standard_B2ms VM costs approximately $0.095/hour while running.

Always delete the resource group at the end of the lab to avoid unexpected charges.

To confirm deletion is complete (takes ~2-3 minutes):

```
az group show --name kafka-lab-rg
```

Should return "ResourceGroupNotFound" once finished.

What You Built

- A Bicep template that provisions a complete VM environment with zero manual installation.
- An Apache Kafka broker running in KRaft mode (no ZooKeeper) as a managed Linux service.
- A .NET 8 WebAPI (PaymentProcessor) that acts as a Kafka producer.
- A .NET 8 console app (PaymentSimulator) with concurrent producer and consumer threads.
- A real-time event streaming pipeline — the foundation of modern payment and notification systems.